

Introduction

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) is the most influential density-based clustering algorithm. Unlike k -means, it does not require specifying the number of clusters in advance, finds clusters of arbitrary shape (rings, half-moons, elongated bands), and explicitly flags low-density points as noise rather than forcing them into a cluster. These properties make it the right choice for spatial data, anomaly detection alongside clustering, and any context where the intuitive notion of “cluster” is “a dense region surrounded by low-density gaps”.

Prerequisites

A working understanding of distance metrics, density-based reasoning, and the limitations of partitioning algorithms (k -means, PAM) for non-convex cluster shapes.

Theory

DBSCAN has two parameters: ϵ (eps), the neighbourhood radius, and minPts, the minimum number of points (including the centre) required for a point to be a *core* point.

- A **core point** has at least minPts points within distance ϵ .
- A **border point** is within ϵ of a core point but is not itself a core point.
- A **noise point** is neither.

Clusters are formed as maximal connected sets of core points (with their associated border points), with noise points left unassigned. The algorithm is order-invariant for core-point structure but border-point assignments can depend on processing order. Choosing ϵ is the practical challenge; the standard heuristic is the k -distance plot, which shows the distance to the k -th nearest neighbour for each point sorted in ascending order — a knee in the curve marks a sensible ϵ .

Assumptions

A meaningful distance metric; clusters have sufficient density separation to be distinguished from noise; standardised input so distances are comparable across variables.

R Implementation

```
library(dbscan)

X <- scale(iris[, 1:4])

# Choose eps from k-distance plot
kNNdistplot(X, k = 5)
abline(h = 0.7, col = "red", lty = 2)

# DBSCAN
db <- dbscan(X, eps = 0.7, minPts = 5)
db$cluster

table(db$cluster, iris$Species)
```

Output & Results

`dbscan()` returns a vector of cluster labels with 0 indicating noise. Core points, border points, and noise are tagged separately in the result object. The k -distance plot guides ϵ selection; the table comparison against species labels shows how DBSCAN’s discovered structure aligns with known classes.

Interpretation

A reporting sentence: “DBSCAN with $\epsilon = 0.7$ and $\text{minPts} = 5$ identified three clusters and flagged 10 observations as noise; the clusters reproduced the iris species structure with 88 % agreement, with the noise points concentrated at the versicolor / virginica boundary.” Always state ϵ , minPts , and the number of noise points; clusters identified are sensitive to all three.

Practical Tips

- Use the k -distance plot (`kNNdistplot()`) to select ϵ ; a clear knee in the sorted distance curve marks a sensible threshold.
- DBSCAN is sensitive to ϵ , especially with clusters of widely different densities; use OPTICS (`dbscan::optics()`) or HDBSCAN (`dbscan::hdbscan()`) when density varies across clusters.
- Standardise variables before clustering; distance-based methods misbehave when scales differ across columns.
- Treat noise points explicitly in downstream analysis; do not force them into the nearest cluster, as that defeats DBSCAN’s main advantage over k -means.
- For high-dimensional data, distances concentrate (the “curse of dimensionality”) and DBSCAN loses discriminating power; reduce dimensionality first via PCA or UMAP.
- HDBSCAN is generally more robust than DBSCAN — it avoids the ϵ choice and handles varying densities — and is increasingly preferred for new applications.

R Packages Used

`dbscan` for `dbscan()`, `optics()`, `hdbscan()`, and the k -distance heuristic; `fpc::dbscan()` for an alternative implementation with similar API; `factoextra::fviz_cluster()` for ggplot-style cluster visualisation; `cluster::clusterboot()` for stability-based validation of the resulting clusters.

For Reviewers

What to look for in a paper using this method.

- **Common misapplications.**
- **Diagnostics that should be reported but often aren’t.**
- **Red flags in tables and figures.**
- **What to verify.**
- **What an adequate Methods paragraph must contain.**

See also — labs in this chapter

- PCA, factor analysis, CCA, LDA
- Clustering: k -means, hierarchical, model-based
- UMAP and t-SNE